```
  user = userDouble
  permissions = new Permissions
  assert.equal visits(database, http, user, permissions), 'private'

fact 'Returns visits for user', ->
  database = databaseDouble
  http = httpDouble
  user = userDouble
  permissions = new Permissions
  assert.equal visits(database, http, user, permissions), 'private'
```

Notice that the dependencies are essentially the same in both cases. If the program has already been structured to have the `makeVisitsForUser` function, then dealing with the dependencies in the test is easier:

```
database = databaseDouble
http = httpDouble
visitsForUser = makeVisitsForUser database, http

fact 'Visits not returned when permissions are private', ->
  user = new User
  privatePermissions = new Permissions private: true
  assert.equal visitsForUser(user, privatePermissions), 'private'

fact 'Visits returned for user', ->
  user = new User
  privatePermissions = new Permissions private: true
  assert.equal visitsForBob(database, http), 'private'
```

This change looks minor at a small scale (and it is), but as a program grows, if it has been composed with functions, then you can avoid some messy dependency situations that are otherwise difficult to solve.

In this section you've learned to deal with program dependencies when writing unit tests. The general approach is to create a test double that takes the place of the dependency during the test. You've also seen the different ways to get doubles where you need them during tests. By forcing yourself to write programs that allow dependencies to be replaced by doubles during tests, your programs will be easier to maintain. That's an added benefit that you'll appreciate on another day.

What else do you need to know about testing? This is CoffeeScript, so your applications almost always run in an event-driven environment. One of the